

Fiche TP — Semaine 1

Mise en place du projet

Développement web, classe passerelle

Durée 4h

Rendu avant S2

Configurer un environnement de développement fonctionnel et initialiser le dépôt Git du projet fil rouge **MiniForum**, avec un `.gitignore` correct dès le premier commit.

Ce TP suppose que les notions du polycopié *S01_polycop* (client / serveur, HTML / CSS / JS, Git) ont été vues en cours. Garde-le à portée de main.

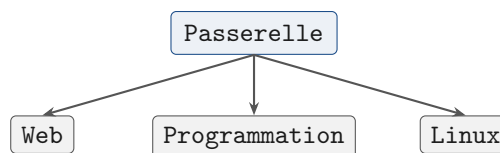
Les instructions ci-dessous concernent **Windows**. Sous macOS ou Linux, les téléchargements sont différents mais les commandes du terminal sont identiques : signale-le et je te guiderai.

1 S'organiser : arborescence et sauvegarde

Avant d'installer quoi que ce soit, on met en place une organisation de fichiers claire et une sauvegarde automatique. Tu ne perdras jamais ton travail, et tu retrouveras toujours facilement tes cours et projets.

Créer l'arborescence des cours

1. Sur ton disque, ouvrir le répertoire Documents.
2. Y créer un répertoire Passerelle.
3. À l'intérieur de Passerelle, créer les trois sous-répertoires suivants :

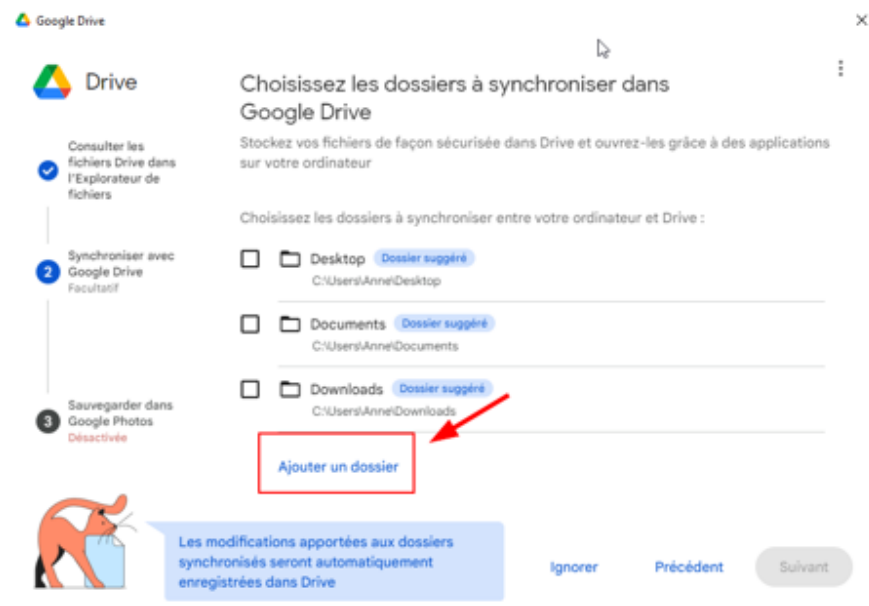


Synchroniser automatiquement sur Google Drive

Afin de ne pas perdre ton travail en cas de perte, de casse ou d'écrasement de disque, tu vas synchroniser automatiquement le répertoire Passerelle sur ton espace Google Drive personnel.

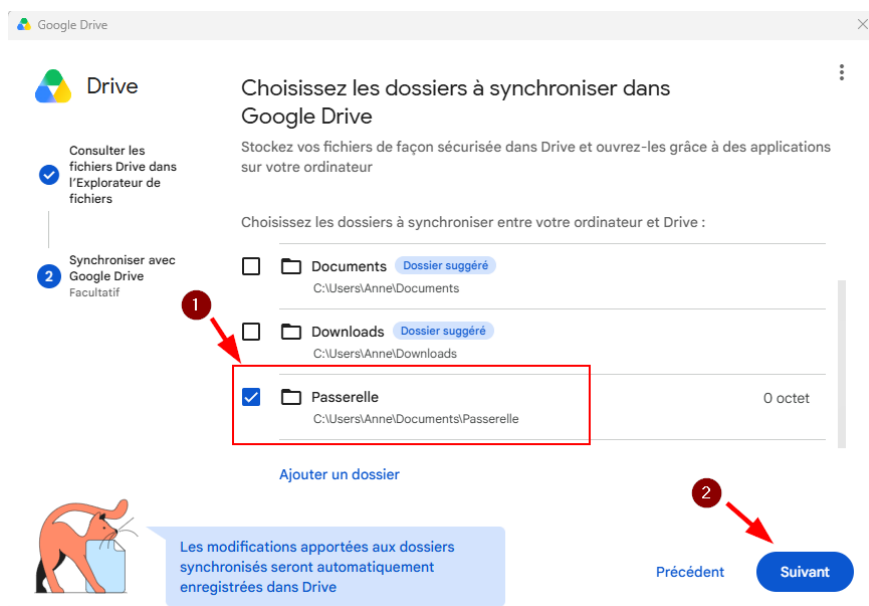
Étape 1 — Mettre en place la synchronisation

1. Télécharger et installer **Drive pour ordinateur** : <https://workspace.google.com/intl/fr/products/drive/>
2. Se connecter avec ton **compte eduge**.
3. Appuyer deux fois sur *Suivant*, jusqu'à l'étape de sélection du dossier à synchroniser.
4. Cliquer sur **Ajouter un dossier**.



Capture d'écran — ajouter un dossier à synchroniser

5. Sélectionner le dossier Passerelle créé à l'étape précédente.
6. Vérifier que **seul** le dossier Passerelle est coché, puis cliquer sur *Suivant*.
7. Continuer en appuyant sur *Suivant*.



Capture d'écran — dossier Passerelle sélectionné

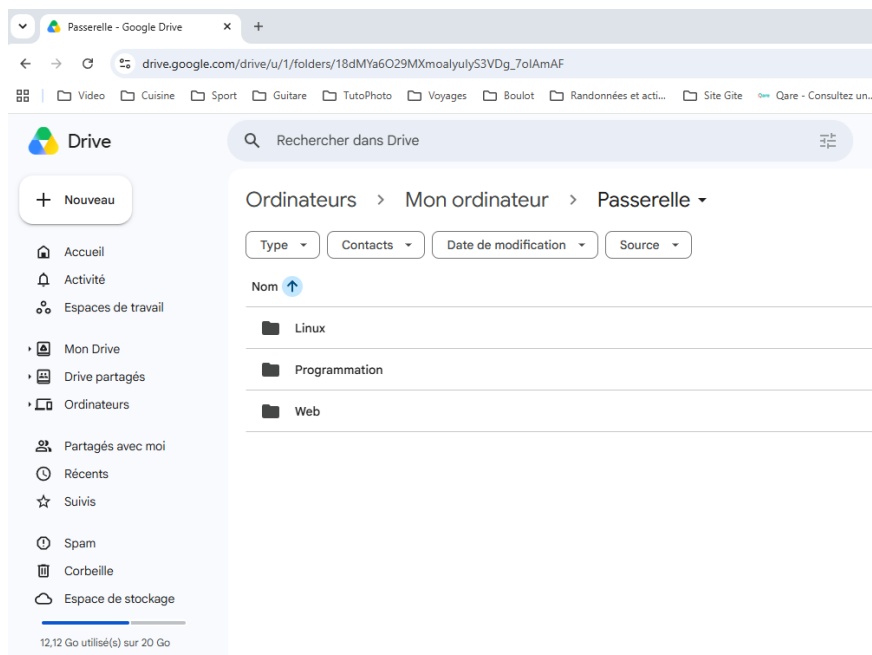
Attention

Ne synchronise que le dossier Passerelle. Ne coche pas *Bureau*, *Documents* ou *Téléchargements* : tu risquerais de synchroniser bien plus que nécessaire et de saturer ton espace Drive.

Étape 2 — Vérifier le bon fonctionnement

1. Dans ton navigateur, aller sur ton **compte eduge** puis ouvrir **Drive**.
2. Dans le Drive, cliquer sur *Ordinateurs* → *Mon ordinateur* → Passerelle.

3. Tu dois retrouver l'arborescence créée sur ton ordinateur.



Capture d'écran — vérification de l'arborescence côté Drive

4. Créer un fichier `test.txt` sur ton ordinateur, dans le dossier `Web`.

5. Vérifier qu'il apparaît bien côté Drive au bout de quelques secondes, puis le supprimer.

Ton travail est désormais sauvegardé automatiquement. Attention toutefois : Google Drive est une **sauvegarde**, pas un outil de versionnage. C'est le rôle de Git, que tu mettras en place en section 4.

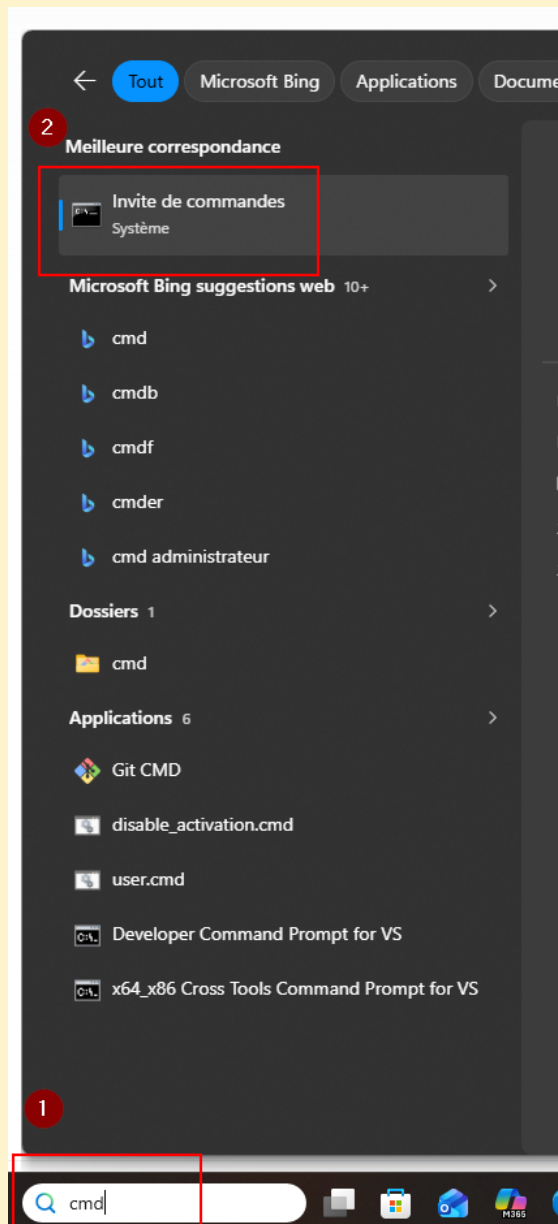
2 Installation des outils (Windows)

Node.js

1. Aller sur <https://nodejs.org/fr/download> et télécharger la version **24 LTS** pour Windows (fichier `.msi`).
2. Lancer l'installateur téléchargé, accepter la licence, cliquer sur *Next*.
3. Dossier de destination : garder celui proposé par défaut.
4. Écran *Custom Setup* : laisser toutes les cases cochées, en particulier **npm package manager** et **Add to PATH**.
5. Écran *Tools for Native Modules* : laisser décoché (non nécessaire pour ce cours), cliquer sur *Next*.
6. Cliquer sur *Install*, patienter, puis *Finish*.

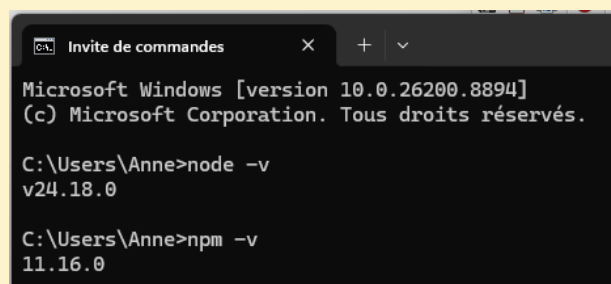
Vérification

1. Ouvrir une invite de commande (*cmd* ou *PowerShell*).



Capture d'écran — ouvrir une invite de commande

2. Exécuter `node -v` puis `npm -v` : chaque commande doit afficher un numéro de version, et celui de Node doit commencer par v24.



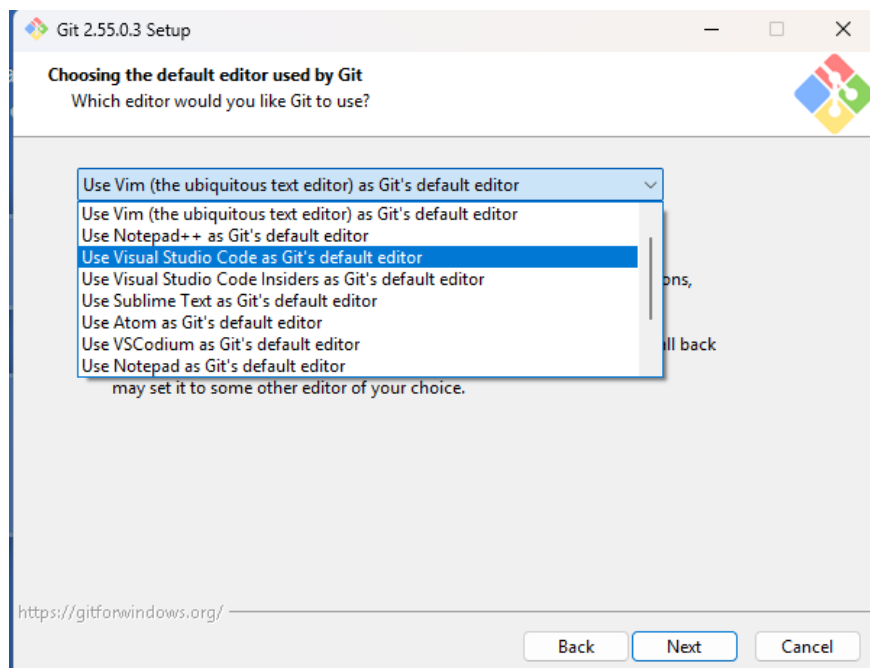
Capture d'écran — vérification de l'installation de Node.js

Ne pas installer Node.js 20

Beaucoup de tutoriels en ligne proposent encore Node.js 20 : cette version est en fin de vie et ne doit plus être utilisée. Si `node -v` affiche autre chose que `v24....`, désinstalle et recommence.

Git

1. Aller sur <https://git-scm.com/install/windows> et télécharger l'installateur proposé (la page détecte automatiquement l'architecture : c'est x64 dans la quasi-totalité des cas).
2. Lancer l'installateur, accepter la licence, garder le dossier de destination par défaut.
3. Écran de sélection des composants : garder les cases cochées par défaut.
4. Écran *Choosing the default editor used by Git* : sélectionner **Visual Studio Code** dans la liste déroulante.



Capture d'écran — choix de l'éditeur par défaut

5. Écran *Adjusting the name of the initial branch* : choisir explicitement **main** comme nom de branche par défaut.
6. Écran *Adjusting your PATH environment* : garder l'option recommandée du milieu (*Git from the command line and also from 3rd-party software*).
7. Écran *Choosing the SSH executable* : garder par défaut (*Use bundled OpenSSH*).
8. Écran *Choosing HTTPS transport backend* : garder par défaut (*Use the OpenSSL library*).
9. Écran *Configuring the line ending conversions* : garder l'option par défaut (*Checkout Windows-style, commit Unix-style line endings*).
10. Écrans suivants (*terminal emulator*, *git pull*, *credential helper*) : garder les options par défaut proposées.
11. Cliquer sur *Install*, puis *Finish*.

Vérification

Ouvrir une invite de commande et exécuter `git --version` : la commande doit afficher un numéro de version.

Visual Studio Code

1. Aller sur <https://code.visualstudio.com> et télécharger la version Windows.
2. Lancer l'installateur, accepter la licence, garder le dossier de destination par défaut.
3. Écran *Select Additional Tasks* : cocher **Add to PATH**, **Register Code as an editor for supported file types**, ainsi que les options *Open with Code* du menu contextuel.
4. Cliquer sur *Install*, puis *Finish* en laissant *Launch Visual Studio Code* coché.

Extensions VS Code

Dans VS Code, ouvrir le panneau des extensions (`Ctrl+Shift+X`) et installer :

Extension	Utilité
Prettier	Formatage automatique du code
ESLint	Détection d'erreurs JavaScript
Live Server	Rechargement automatique du navigateur
GitLens	Visualiser l'historique Git dans l'éditeur

Deux autres extensions seront installées le moment venu : **SQLTools** en semaine 8 (connexion à MySQL) et **REST Client** en semaine 13 (test des routes d'API). Inutile de les installer aujourd'hui.

Désinstaller les extensions d'assistance IA

Étape obligatoire du cours

Les assistants de génération de code sont **interdits** dans ce cours. Leur absence est vérifiée à chaque jalon. Les raisons sont expliquées en section 7 du polycopié.

1. Dans VS Code, ouvrir le panneau des extensions (`Ctrl+Shift+X`).
2. Dans la barre de recherche, taper successivement : `copilot`, `ai`, `codeium`, `tabnine`.
3. Pour chaque extension **installée** qui apparaît (GitHub Copilot, GitHub Copilot Chat, ou équivalent), cliquer sur l'engrenage puis *Désinstaller*.
4. Redémarrer VS Code.
5. Vérifier : dans la vue *Extensions* → *Installed*, plus aucun assistant de code ne doit être présent.

Complétion et assistant, ce n'est pas la même chose

La complétion automatique native de VS Code — celle qui propose `addEventListener` quand tu tapes `addEv` — reste active et n'est pas concernée. Elle se contente de compléter des noms existants ; elle n'écrit pas de code à ta place.

3 Mise en place du projet

1. **Vérifier les installations.** Ouvrir un terminal et exécuter :

```
node -v
git --version
```

Chaque commande doit afficher un numéro de version. Sinon, reprendre la section 2 avant de continuer.

2. **Créer le dossier du projet.** Dans le dossier Web créé précédemment (Documents\Passerelle\Web), créer un dossier nommé miniforum.
3. **Ouvrir le dossier dans VS Code** : menu *Fichier* → *Ouvrir le dossier*, puis sélectionner miniforum.
4. **Créer les trois fichiers de base** — `index.html`, `style.css` et `app.js` — directement depuis l'explorateur de VS Code.
5. **Reproduire l'exemple minimal** vu en cours. Recopie-le à la main : c'est en tapant que l'on retient la syntaxe.

```
1 <!DOCTYPE html>
2 <html lang="fr">
3 <head>
4   <meta charset="UTF-8">
5   <title>MiniForum</title>
6   <link rel="stylesheet" href="style.css">
7 </head>
8 <body>
9   <h1>Bienvenue sur MiniForum</h1>
10  <button id="btn">Cliquez-moi</button>
11  <script src="app.js"></script>
12 </body>
13 </html>
```

Extrait 1 – `index.html`

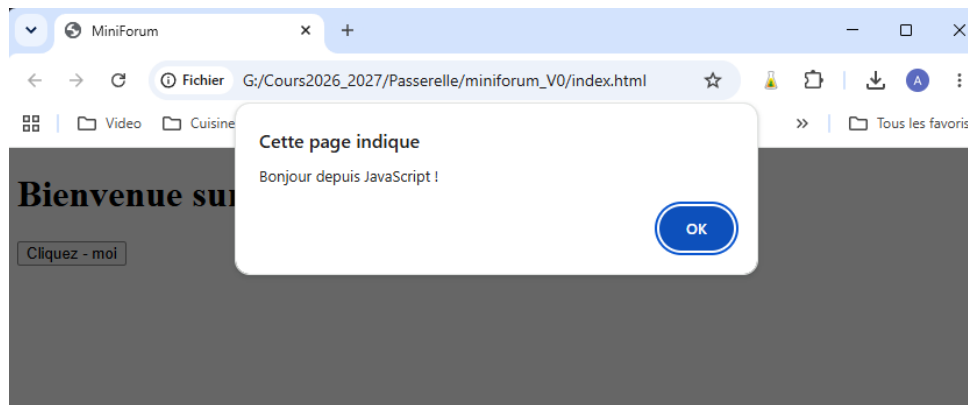
```
1 button {
2   background-color: #1D4E89;
3   color: white;
4   padding: 10px 20px;
5   border: none;
6   border-radius: 4px;
7   cursor: pointer;
8 }
```

Extrait 2 – `style.css`

```
1 const btn = document.getElementById('btn');
2
3 btn.addEventListener('click', () => {
4   alert('Bonjour depuis JavaScript !');
5 });
```

Extrait 3 – `app.js`

6. **Tester dans le navigateur.** Clic droit sur `index.html` dans VS Code → *Open with Live Server*. Le clic sur le bouton doit déclencher une alerte.



Capture d'écran — toute première version de MiniForum

Ouvrir les outils de développement du navigateur (F12), onglet *Console*. Le message d'erreur t'indique la ligne fautive. Les deux causes les plus fréquentes : un nom de fichier mal orthographié dans `<script src="...">`, ou un id qui ne correspond pas entre le HTML et le JavaScript.

4 Git et GitHub

Les commandes de cette section sont à saisir dans **Git Bash** (clic droit dans le dossier miniforum → *Open Git Bash here*).

Configuration de l'identité Git

À la toute première utilisation de Git sur une machine, il faut indiquer qui tu es : ce nom et cet e-mail seront associés à chacun de tes commits.

```
git config --global user.name "Prenom Nom"
git config --global user.email "ton.email@example.com"
```

Cette configuration ne se fait **qu'une seule fois par machine** : elle vaudra pour tous tes futurs projets Git, pas seulement MiniForum.

Créer le fichier .gitignore

À faire avant le premier commit

Un fichier déjà enregistré dans un commit reste dans l'historique **même si on l'ajoute au .gitignore par la suite**. Le .gitignore se crée donc **maintenant**, avant tout `git add`.

Dans VS Code, créer à la racine de miniforum un fichier nommé exactement .gitignore (avec le point initial, et sans extension), puis y copier :

```
1 # Dépendances Node.js : jamais dans Git
2 node_modules/
3
4 # Variables d'environnement (mots de passe, clés d'API)
5 .env
6
7 # Fichiers système
```

```
8 .DS_Store
9 Thumbs.db
10
11 # Réglages locaux de l'éditeur
12 .vscode/
```

Extrait 4 – .gitignore

Premier commit local

```
cd ~/Documents/Passerelle/Web/miniforum
git init
git branch -M main
git status
git add .
git commit -m "Structure de base du MiniForum"
```

Avant le `git add`, `git status` liste les fichiers *non suivis*. Tu dois y voir `index.html`, `style.css`, `app.js` et `.gitignore` — et rien d'autre. Prends l'habitude de lire cette sortie avant chaque commit : c'est ce qui évite d'envoyer par erreur un fichier qui n'a rien à y faire.

Créer un compte GitHub

Si ce n'est pas déjà fait : aller sur <https://github.com>, cliquer sur *Sign up* et suivre les étapes (e-mail, mot de passe, nom d'utilisateur).

Créer le dépôt distant

1. Se connecter sur github.com, cliquer sur le bouton **+** en haut à droite, puis *New repository*.
2. *Repository name* : `miniforum`.
3. Laisser la visibilité en *Public*.
4. **Ne cocher aucune case** (*Add a README file*, *.gitignore*, *licence*) : le dépôt doit rester **vide**, puisque le projet existe déjà sur ta machine.
5. Cliquer sur *Create repository*.

Piège fréquent

Si tu coches *Add a README file*, GitHub crée un premier commit sur le dépôt distant. Ton envoi (`git push`) sera alors refusé, car le dépôt local et le dépôt distant auront des historiques différents. Le plus simple : supprimer le dépôt et recommencer avec un dépôt vraiment vide.

Envoyer le projet sur GitHub

Sur la page du dépôt fraîchement créé, cliquer sur le bouton vert **Code**, onglet **HTTPS**, et copier l'URL affichée. Puis, dans Git Bash, à la racine de `miniforum` :

```
git remote add origin https://github.com/ton-login/miniforum.git
git push -u origin main
```

Lors du tout premier `git push`, une fenêtre s'ouvre dans le navigateur et demande de se connecter à GitHub. C'est normal : GitHub n'accepte plus les mots de passe directement dans le terminal. Il suffit de se connecter une fois — Git mémorise ensuite la connexion.

Vérification

Rafraîchir la page du dépôt sur `github.com`. Les quatre fichiers doivent y apparaître : `index.html`, `style.css`, `app.js` et `.gitignore`.

Le dépôt `miniforum` va évoluer chaque semaine, et son **historique fait partie de l'évaluation du projet final**. Prends dès maintenant l'habitude de committer à chaque étape logique de ton travail, avec un message clair — et non en une seule fois à la fin.

5 Points de contrôle

- ☐ Le dossier `Documents\Passerelle` est créé et synchronisé sur Drive
- ☐ `node -v` affiche une version `v24...` et `git --version` affiche un numéro
- ☐ Les extensions Prettier, ESLint, Live Server et GitLens sont installées
- ☐ **Aucune extension d'assistance IA n'est installée dans VS Code**
- ☐ Le dossier `miniforum` contient `index.html`, `style.css`, `app.js` et `.gitignore`
- ☐ Le clic sur le bouton déclenche bien l'alerte JavaScript
- ☐ L'identité Git (`user.name` et `user.email`) est configurée
- ☐ Le dépôt Git local contient au moins un commit, sur la branche `main`
- ☐ Le dépôt GitHub a été créé **vide** (sans README)
- ☐ Les quatre fichiers sont visibles sur GitHub

À rendre avant la semaine 2

Le lien du dépôt GitHub `miniforum`, envoyé par courriel ou déposé sur la plateforme du cours.

Ce TP est une mise en place, pas une évaluation : personne n'est censé y arriver seul du premier coup. Si une étape résiste plus de dix minutes, demande — c'est exactement à cela que sert la séance.